Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни

«Об’єктно-орієнтоване програмування»

на тему

Розробка графічного редактора об’єктів

Виконав
студент групи ІМ-о51
Козорог Єгор Євгенійович
Номер в списку - 3

Перевірив
Порєв В.М.

Київ 2026

Завдання:

1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту:

Ліміт елементів: 103
Тип масиву: Динамічний
Пунктирна лінія чорного кольору для варіантів
Увід прямокутника: від центру до одного з кутів для варіантів
Відображення прямокутника: чорний контур прямокутника без заповнення
Увід еліпсу: по двом протилежним кутам охоплюючого прямокутника для варіантів
Відображення еліпсу: чорний контур з кольоровим заповненням
Кольори заповнення еліпсу: рожевий
Позначка поточного типу об’єкту, що вводиться: в заголовку вікна

Код:

Form1.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;

namespace Lab2
{
    public partial class Form1 : Form
    {
        private const int N = 103;
        private List<Shape> pcshape = new List<Shape>();

        private enum ToolType { Point, Line, Rectangle, Ellipse }
        private ToolType currentTool = ToolType.Point;

        private bool isDrawing = false;
        private Point startPoint;
        private Point currentPoint;

        public Form1()
        {
```

```
    InitializeComponent();
    this.DoubleBuffered = true;
    BuildMenu();
    UpdateWindowTitle();
}

private void BuildMenu()
{
    MenuStrip menuStrip = new MenuStrip();
    ToolStripMenuItem objectsMenu = new ToolStripMenuItem("Об’єкти");
    objectsMenu.DropDownItems.Add("Крапка", null, (s, e) => SelectTool(ToolType.Point));
    objectsMenu.DropDownItems.Add("Лінія", null, (s, e) => SelectTool(ToolType.Line));
    objectsMenu.DropDownItems.Add("Прямокутник", null, (s, e) => SelectTool(ToolType.Rectangle));
    objectsMenu.DropDownItems.Add("Еліпс", null, (s, e) => SelectTool(ToolType.Ellipse));

    menuStrip.Items.Add(objectsMenu);

    this.MainMenuStrip = menuStrip;
    this.Controls.Add(menuStrip);
}

private void SelectTool(ToolType tool)
{
    currentTool = tool;
    UpdateWindowTitle();
}

private void UpdateWindowTitle()
{
    string toolName = "";
    switch (currentTool)
    {
        case ToolType.Point: toolName = "Крапка"; break;
        case ToolType.Line: toolName = "Лінія"; break;
        case ToolType.Rectangle: toolName = "Прямокутник"; break;
        case ToolType.Ellipse: toolName = "Еліпс"; break;
    }
    this.Text = $"Поточний об'єкт: {toolName}";
}

protected override void OnMouseDown(MouseEventArgs e)
{
    base.OnMouseDown(e);
    if (e.Button == MouseButtons.Left)
    {
        isDrawing = true;
        startPoint = e.Location;
        currentPoint = e.Location;
    }
}

protected override void OnMouseMove(MouseEventArgs e)
{
    base.OnMouseMove(e);
    if (isDrawing)
    {
        currentPoint = e.Location;
        this.Invalidate();
```

```
    }
}

protected override void OnMouseUp(MouseEventArgs e)
{
    base.OnMouseUp(e);
    if (isDrawing && e.Button == MouseButtons.Left)
    {
        isDrawing = false;

        if (pcshape.Count < N)
        {
            Shape newShape = FactoryCreateShape(currentTool);
            if (newShape != null)
            {
                newShape.SetCoords(startPoint.X, startPoint.Y, e.X, e.Y);
                pcshape.Add(newShape);
            }
        }
        else
        {
            MessageBox.Show($"Досягнуто ліміт об'єктів N = {N}!", "Увага",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }

        this.Invalidate();
    }
}

private Shape FactoryCreateShape(ToolType tool)
{
    switch (tool)
    {
        case ToolType.Point: return new PointShape();
        case ToolType.Line: return new LineShape();
        case ToolType.Rectangle: return new RectangleShape();
        case ToolType.Ellipse: return new EllipseShape();
        default: return null;
    }
}

protected override void OnPaint(PaintEventArgs e)
{
    base.OnPaint(e);
    Graphics g = e.Graphics;
    g.SmoothingMode = SmoothingMode.AntiAlias;

    foreach (var shape in pcshape)
    {
        shape.Draw(g);
    }

    if (isDrawing)
    {
        using (Pen rubberPen = new Pen(Color.Black, 1) { DashStyle = DashStyle.Dash })
        {
            int x1 = startPoint.X;
            int y1 = startPoint.Y;
```

```
                int x2 = currentPoint.X;
                int y2 = currentPoint.Y;

                switch (currentTool)
                {
                    case ToolType.Point:
                        g.DrawLine(rubberPen, startPoint, currentPoint);
                        break;
                    case ToolType.Line:
                        g.DrawLine(rubberPen, startPoint, currentPoint);
                        break;
                    case ToolType.Rectangle:
                        int rx = Math.Abs(x2 - x1);
                        int ry = Math.Abs(y2 - y1);
                        g.DrawRectangle(rubberPen, x1 - rx, y1 - ry, 2 * rx, 2 * ry);
                        break;
                    case ToolType.Ellipse:
                        int left = Math.Min(x1, x2);
                        int top = Math.Min(y1, y2);
                        int width = Math.Abs(x2 - x1);
                        int height = Math.Abs(y2 - y1);
                        g.DrawEllipse(rubberPen, left, top, width, height);
                        break;
                }
            }
        }
    }
}
```

Shape.cs

```
using System;
using System.Drawing;

namespace Lab2
{
    public abstract class Shape
    {
        protected int x1, y1, x2, y2;

        public void SetCoords(int x1, int y1, int x2, int y2)
        {
            this.x1 = x1;
            this.y1 = y1;
            this.x2 = x2;
            this.y2 = y2;
        }

        public abstract void Draw(Graphics g);
    }

    public class PointShape : Shape
    {
```

```
        public override void Draw(Graphics g)
        {
            using (SolidBrush brush = new SolidBrush(Color.Black))
            {
                g.FillRectangle(brush, x1, y1, 2, 2);
            }
        }
    }

    public class LineShape : Shape
    {
        public override void Draw(Graphics g)
        {
            using (Pen pen = new Pen(Color.Black, 1))
            {
                g.DrawLine(pen, x1, y1, x2, y2);
            }
        }
    }

    public class RectangleShape : Shape
    {
        public override void Draw(Graphics g)
        {
            using (Pen pen = new Pen(Color.Black, 1))
            {
                int rx = Math.Abs(x2 - x1);
                int ry = Math.Abs(y2 - y1);

                int left = x1 - rx;
                int top = y1 - ry;
                int width = 2 * rx;
                int height = 2 * ry;

                g.DrawRectangle(pen, left, top, width, height);
            }
        }
    }

    public class EllipseShape : Shape
    {
        public override void Draw(Graphics g)
        {
            int left = Math.Min(x1, x2);
            int top = Math.Min(y1, y2);
            int width = Math.Abs(x2 - x1);
            int height = Math.Abs(y2 - y1);

            using (SolidBrush brush = new SolidBrush(Color.Pink))
            using (Pen pen = new Pen(Color.Black, 1))
            {
                g.FillEllipse(brush, left, top, width, height);
                g.DrawEllipse(pen, left, top, width, height);
            }
        }
    }
}
```

```
```